

● FinMind

● TEJ

● DuckDB

financial-db 台股研究級資料倉儲

以 Parquet + DuckDB 建置兩條可重現的資料管線，並用跨來源驗證找出資料與程式的錯誤

一頁看懂：建了什麼、驗證了什麼

●
3,394 萬

列日資料：兩個來源的行情與指數成分，最早自 1994 年 10 月

●
2 條管線

FinMind API 與 TEJ Excel 增益集，共用三層契約，對外提供 39 個 view

●
868 萬

筆日報酬做跨來源比對，99.96% 相差在 10bp 以內

●
286 + 63

個自動化測試與健康檢查，每次改動後全部重跑

● 建了什麼

把上游資料原樣落地 (raw)，整理成有型別、去重、分區的 Parquet (processed)，再用一張資料表都沒有的 DuckDB view 提供查詢。

● 怎麼確保正確

每一層都能由上一層重建；文件與斷言裡的數字一律實測；用對帳 view、健康檢查與第二個資料來源交叉驗證。

● 發現了什麼

FinMind 的還原價在上市股 2003 年前、上櫃股 2008 年前其實沒有還原；同一套比對也抓到並修正了自己管線的一個去重錯誤。

為什麼不直接呼叫 API：回測資料的三個陷阱

直接拿公開資料做回測，結果常常好得不真實。這個專案把三個常見問題變成設計需求。

● 存活者偏誤

問題

證券主檔只列現在還在的股票，已下市的公司會從歷史橫斷面中消失，報酬因此被高估。

設計回應

每月抽一天掃描全市場，找回 767 檔已下市證券；TEJ 側保留 462 檔已停止交易公司的完整歷史。

● 還原價會被改寫

問題

每次除權息都會重寫整段還原歷史。只會往後追加的資料庫，舊資料會悄悄過期。

設計回應

每週全量重抓還原價，並把「這筆資料屬於哪個還原基準」納入去重規則。

● 單一來源無法自證

問題

單位錯誤、缺漏、沒調整到的除權息——來源自己的錯，只看這個來源是看不出來的。

設計回應

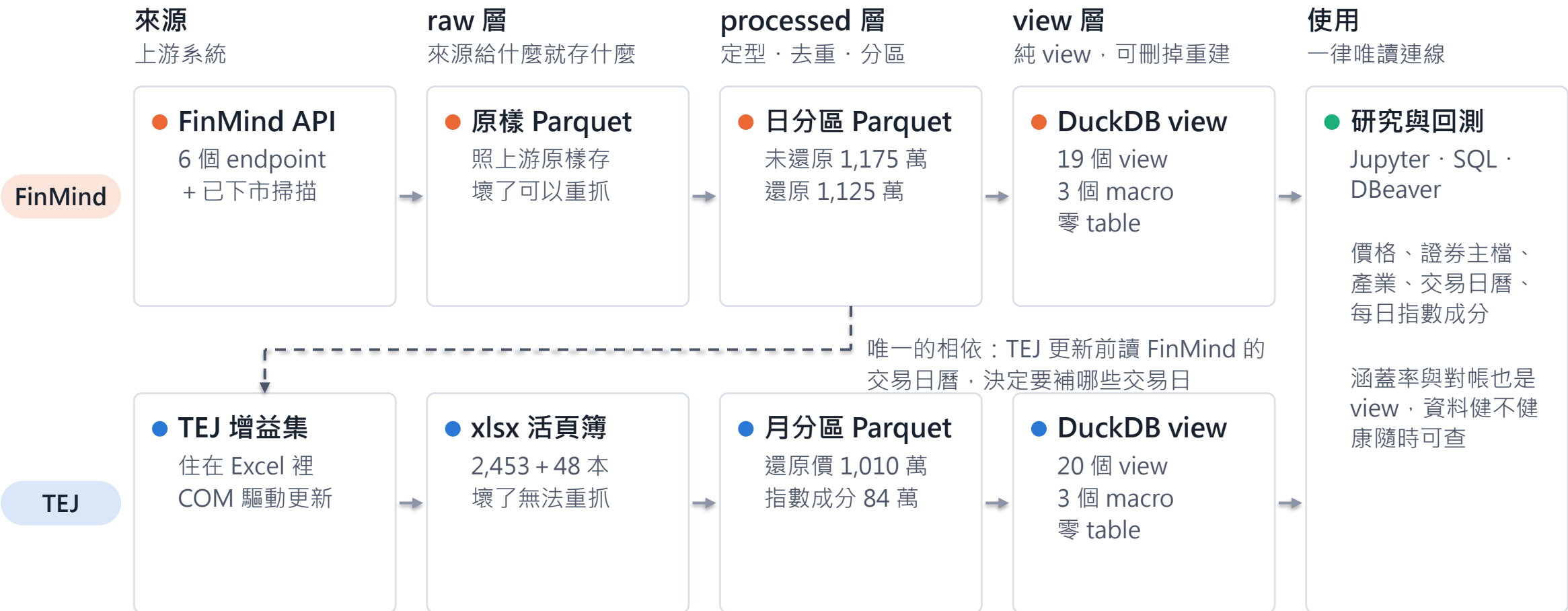
兩個獨立來源、同一套契約，逐日比對報酬，再用漲跌幅限制檢驗合理性。

一 架構與資料

三層契約、Parquet + DuckDB 的取舍、資料盤點

三層契約：兩條管線，同一個心智模型

raw 保留上游原貌，processed 才是查詢契約，DuckDB 只是蓋在 Parquet 上的 view。



為什麼是 Parquet + DuckDB：每個取捨都有實測數字

- Parquet 是唯一真相

欄式、壓縮、任何工具都讀得到。DuckDB 檔裡一張資料表都沒有，刪掉重跑一支程式就回來，也避開了單一寫入者的檔案鎖。

- 分區粒度由更新方式決定

FinMind 每天只新增一個檔，所以一天一檔、寫入天然冪等；TEJ 每次全量重建，改用一月一檔。

- 日期查詢一律走 macro

分區鍵是 year / month，只寫日期條件不會裁剪檔案。*_between(lo, hi) macro 會自動補上 year 條件。



268 KB

兩個 .duckdb 檔各自的大小：純 view、零 table



4.3×

補上分區條件後，查一季資料由 0.56 秒降到 0.13 秒



15×

TEJ 月分區相對日分區的全表掃描 (624 ms → 41 ms)



2.8×

同一份資料，月分區的檔案較小 (188.9 MB → 67.1 MB)

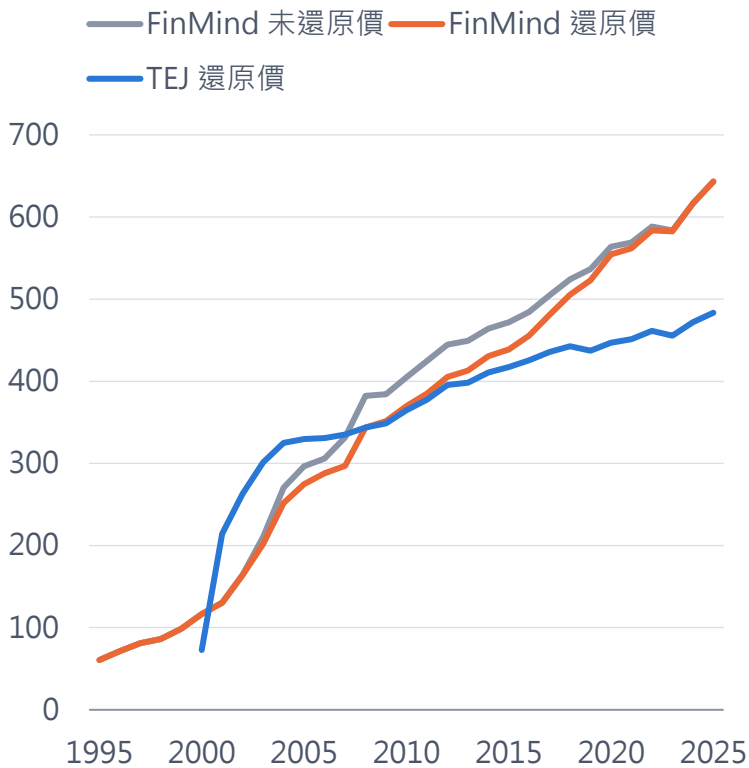
有哪些資料：3,394 萬列、五類資料集

資料集	來源	列數	證券數	期間
未還原日行情 開高低收、成交量值、筆數	FinMind	11,751,428	3,614	1994-10 → 2026-09
還原日行情	FinMind	11,252,462	3,154	1994-10 → 2026-09
還原日行情 40 欄：估值、權重、處置旗標	TEJ	10,101,676	2,453	2000-09 → 2026-10
指數每日成分 TWN50、TM100	TEJ	835,348	367	2002-07 → 2026-10
維度：證券主檔、產業鏈、 交易日曆、已下市清單	FinMind	3,916 檔證券 8,141 個交易日	—	1994-10 起

沒有抓的：財報、三大法人、融資券、股利明細、盤中與分鐘資料。

FinMind 的價格範圍刻意排除權證與 TDR：全市場一天約 4.4 萬檔，只保留約 2,800 檔。

每年列數（千列）



未滿一年的 2026 年不列入。

兩個來源各有所長：先選對資料庫

面向	FinMind	TEJ
來源與取得	FinMind API v4 (HTTP + token) ，隨時可以重抓	TEJ Smart Wizard Excel 增益集；授權資料，raw 無法重製
涵蓋	全市場 3,916 檔：上市、上櫃、興櫃、ETF、指數	公司池 2,453 檔，加上 TWN50 / TM100 每日成分
期間	1994-10-01 起	2000-09-04 起
已下市公司的還原價	找回的 767 檔中只有 7 檔有 (上游缺口)	462 檔已停止交易者全數保留
每列欄位	精簡：開高低收、成交量值、筆數	40 欄：估值、殖利率、市值比重、漲跌停與處置旗標
適合	全市場橫斷面、選股池、產業、2000 年以前	無偏誤的還原報酬、估值因子、指數的歷史成分

二 蒐集、處理與更新

API 與 Excel 兩種截然不同的來源，如何收斂到同一套流程

FinMind：用最少的 API 呼叫抓完 32 年



● 兩種抓取形狀

回補時一次呼叫拿一檔的全部歷史，約 3,600 次呼叫補完 32 年；每日更新時一次呼叫拿全市場一天。兩種都原樣落地，raw 只增不改。

● 跨行程限流

自寫 token-bucket 限流器，狀態存在檔案裡，排程與手動執行共用同一份配額（方案上限的 90%）。配額用盡與伺服器錯誤各有退避重試。

● 找回已下市證券

主檔只列現存證券。自 2004 年起每月抽一天掃描全市場（271 次呼叫），再與下市公告聯集：只靠掃描找到 296 檔、只靠公告 320 檔、兩者都有 151 檔。

TEJ：沒有 API，就把 Excel 變成可排程的資料來源



● 繞不掉的 Excel

唯一能更新資料的是住在 Excel 裡的增益集。程式以 COM 啟動獨立的 Excel，自行載入增益集並呼叫同步巨集；卡住時由看門狗結束該行程。

● 不打擾使用者

增益集更新時會把視窗翻成可見並搶走焦點。把執行緒綁到獨立的 Windows 桌面後，使用者桌面上的視窗從 4 個變成 0 個，耗時 12.4 秒對 13.0 秒。

● 從樣板長出活頁簿

查詢定義不含證券代碼，改寫代碼欄再更新，就得到另一檔的完整歷史。2,300 多本由此自動建立；與手動下載的檔逐格比對，230,796 格全部相同。

raw → processed : 把上游的資料變成查詢契約

● 1 改名與定型

max→high、min→low；日期字串轉 DATE、金額轉 BIGINT (成交值會超過 int32)。stock_id 刻意保留字串：0050、00679B。

● 2 過濾範圍

全市場一天回 43,727 檔，其中約 4 萬檔是權證與 TDR。依代碼形狀只留普通股與 ETF，共 2,821 檔。

● 3 重新分區

把「一檔一個檔案」翻成「一天一個檔案」。約 1,200 萬列，交給 DuckDB 逐年分塊處理，記憶體只需幾十萬列。

● 4 去重

每張表先寫下自然鍵。未還原價以全市場日檔為準；還原價以 ticker 全歷史為準 (原因見第三部分)。

為什麼不用 SELECT DISTINCT *

上游每次重爬都會把日期欄重新蓋章。沒有鍵的去重會把同一筆事實每爬一次多存一列。

6,871 → 11,520

產業表在第二次抓取後的列數 (修正前)

改以 (證券、產業、子產業) 為鍵後回到 6,871 列，並在健康檢查裡加上守門規則。TEJ 側則不去重，直接斷言鍵唯一，違反就讓重建失敗。

Schema (FinMind 價格) : 三層各長什麼樣

raw 保留上游欄名 ; processed 是唯一該查的一層 ; view 只做組合與分區裁剪。

● raw

API 回什麼就存什麼

date (字串)
stock_id
open / max / min / close
spread
Trading_Volume
Trading_money
Trading_turnover

● processed

查詢契約：定型、改名、去重

date (DATE) · stock_id ← 鍵
open / high / low / close
spread
volume
turnover_value
transactions
source · ingested_at
year / month ← 分區鍵

● view

v_daily_prices 等 19 個 view

v_daily_prices : 同 processed 14 欄
v_prices_combined : 加上還原價與還原因子
v_daily_prices_enriched : 加上名稱、產業、下市日
*_between(lo, hi) : 自動裁剪分區

Schema (TEJ 還原價) : 35 欄中文表頭 → 40 欄 view

同一列只有開高低收是還原價，買賣報價與次日參考價是未還原的原始報價，混用必錯。

● raw

xlsx 35 欄，中文表頭逐字驗證

證券代碼 (代碼 + 名稱同格)
年月日 (Excel 序號)
開盤價 / 最高價 / 最低價 / 收盤價
成交量(千股) · 成交值(千元)
本益比 · 股價淨值比 · 殖利率
最後揭示買價 / 賣價
注意 · 處置 · 全額交割旗標

● processed

38 欄：英文欄名、真正的型別

stock_id · date ← 鍵 (斷言唯一)
stock_name (由代碼欄切出)
open / high / low / close (還原)
bid / offer / next_ref_price (未還原)
volume_k_shares · turnover_k
pe_* / pbr_* / div_yield_*
source_file · ingested_at

● view

v_adj_daily_prices 等 20 個 view

價格主表 40 欄 (+ year / month)
v_index_constituents : 每日指數成分
index_members_on(指數, 日期)
v_workbook_status : 活頁簿對帳
v_security_gaps : 存續期內缺漏日

view 層：39 個 view，把「怎麼查才對」寫進資料庫

用途	FinMind (19 view + 3 macro)	TEJ (20 view + 3 macro)
價格	v_daily_prices、v_adj_daily_prices、 v_prices_combined、v_daily_prices_enriched	v_adj_daily_prices (40 欄)、v_securities
回測用的證券範圍	v_securities_all (含已下市)、 v_delisted_securities、v_stock_info_latest	v_index_constituents、v_index_membership、 v_index_universe
產業與日曆	v_industry_by_stock、v_trading_calendar、 v_last_trading_day	v_trading_calendar、v_index_calendar
涵蓋率與對帳	v_daily_coverage、v_missing_trading_days、 v_thin_trading_days、v_known_empty_days	v_workbook_status、v_security_gaps、 v_index_price_gaps、v_thin_index_days
維運與入口	v_ingestion_runs ; daily_prices_between(lo, hi)	v_ingestion_runs ; index_members_on(指數, 日期)

涵蓋率與對帳本身就是 view：資料健不健康，用一句 SQL 就能問。回測歷史指數時，用 index_members_on() 取當日成分，而不是把所有曾入選的股票放進每一天。

如何更新：每日增量、每週全量、漏跑自動補

	週一	週二	週三	週四	週五	週六	週日
● FinMind 每日更新 3 個維度 + 缺漏日；秒級	18:30	18:30	18:30	18:30	18:30	18:30	
● FinMind 每週重抓 還原價全量 + 對帳；約 45 分							02:00
● TEJ 每日更新 指數檔 + 約 150 本；約 55 分	18:45	18:45	18:45	18:45	18:45	18:45	18:45
● TEJ 每週全量 2,453 本全部重抓；約 7 小時							03:00
	☒ 已註冊的排程		☐ 程式已完成、排程尚未註冊				

● 每日抓三個集合

新日期 U 最近 3 天重抓 U 範圍內缺漏。上游收盤後會修正當日成交量，漏跑的日子下次自動補齊。

● 還原價每週全量重抓

除權息會改寫整段歷史，日期補齊不等於數字正確。

● 寧可停下，不寫半筆

TEJ 新增的日期沒有收盤價就不存檔並中止；重建先寫暫存目錄，再原子換檔。

● 現況 (2026-10-04)

FinMind 的 token 自 9 月 19 日降為免費方案，排程失敗、資料停在 9 月 16 日；每次失敗都留在 v_ingestion_runs。

把「資料正確」做成可以重跑的檢查

●
286

個自動化測試 (FinMind 210 、
TEJ 76)

●
63

項唯讀健康檢查 (28 + 35) ，排
程後自動執行

●
4 類

每週對帳：缺漏日、稀薄日、沒
有資料的證券、停更的證券

●
0

次手改資料：資料只由程式產生，
要改資料就改程式再重跑

處理過的上游缺陷 (各有對應測試)

日期欄是字串「None」(32 列)

1999 年以前有週六盤 (176 天)

官方日曆列了實際休市的日子 (2026-07-10)

收盤後成交量被修正 (0050 : 39,946,000 → 43,382,378)

產業名稱飄移 (創新板 / 創新版等)

處理方式

轉成真正的 NULL，否則轉型會直接失敗

不寫任何「週末必無交易」的邏輯

向 API 確認一次後記錄下來，不再當成缺漏反覆重抓

每日重抓最近 3 個交易日

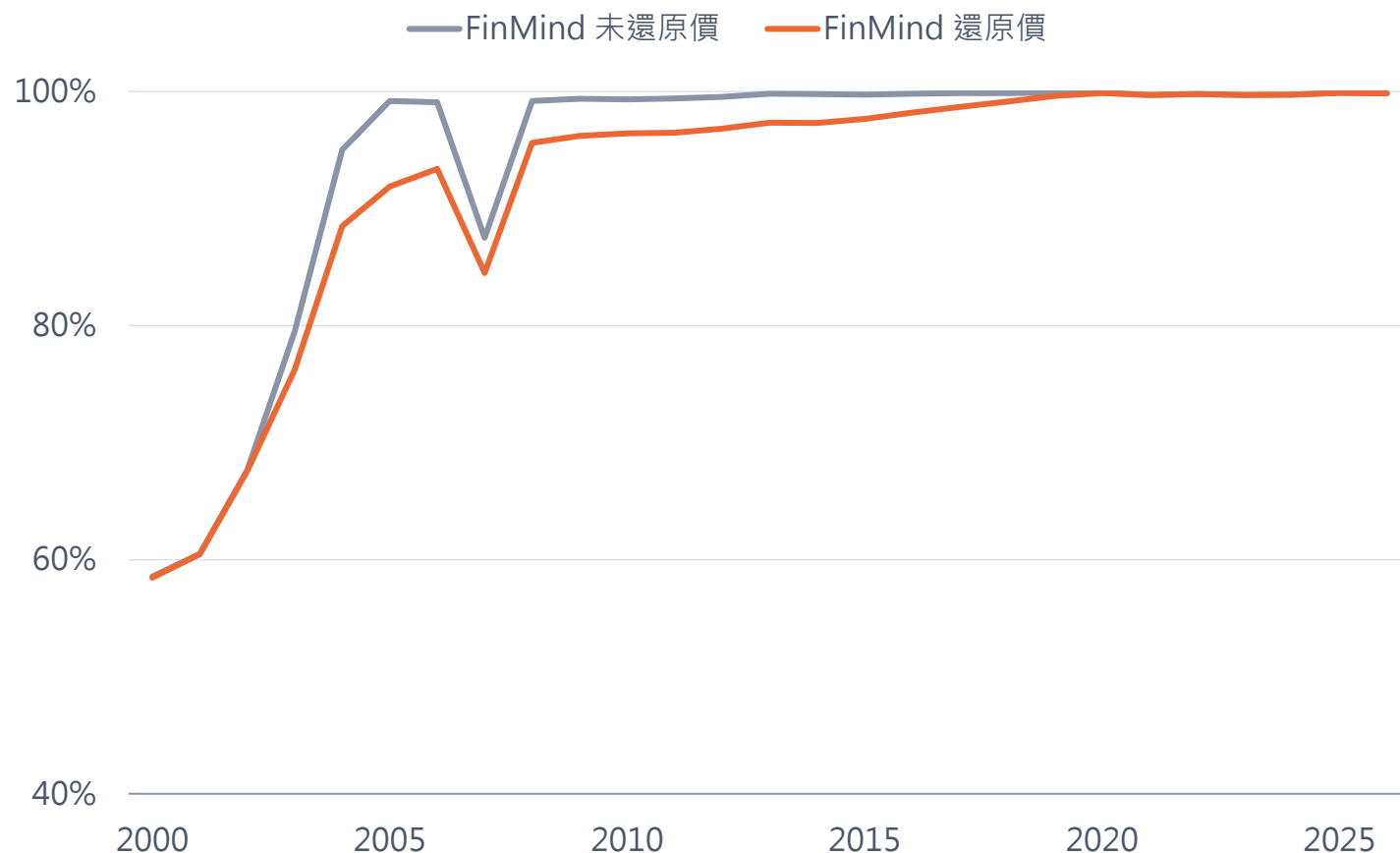
別名表正規化，原值另存

三 跨來源驗證

FinMind 與 TEJ 哪裡不同、誰是對的、各自哪裡不可信

存活者偏誤：兩個來源保留了多少已下市公司

回測只用現存公司，報酬會被高估。下圖是 TEJ 上市櫃資料列中，FinMind 也有的比例。



7 / 767

FinMind 找回的已下市證券中有還原價的檔數 (未還原價有 466 檔)



462

TEJ 已停止交易、仍保有完整還原歷史的檔數；
FinMind 還原價只涵蓋其中 197 檔



2004-02-11

FinMind 能做無偏誤橫斷面的最早日期 (全市場端點的資料起點)

驗證方法：比報酬，不比價格水準

● 1 為什麼不比水準

兩邊都把歷史回溯到最新基準，只要重抓的日期不同，整段水準就差一個常數。逐列比收盤價，只有 69.3% 相差在 0.01 元以內——但那不代表有錯。

● 2 改比日報酬

同一檔、同一天的報酬不受基準影響。再拿未還原報酬當第三把尺：誰的報酬等於未還原報酬，誰那天就沒有調整。

● 3 用交易制度當檢驗

台股有漲跌幅限制（2015 年 6 月前 7%，之後 10%）。還原後仍超過限制的報酬，不是真實事件，就是沒有調乾淨。

●
2,188 檔

兩邊共有的證券，交集 941 萬列

●
868 萬

筆上市櫃、且前後兩天都有成交的日報酬

●
99.96%

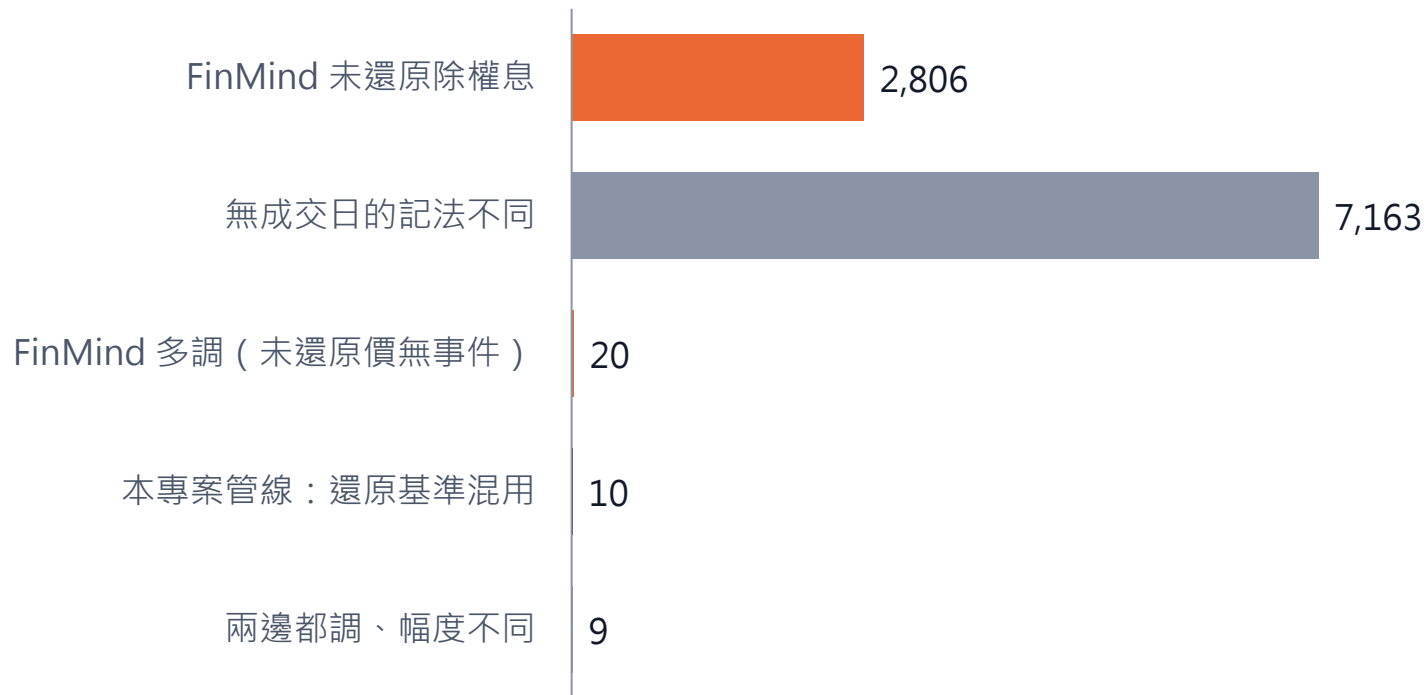
的日報酬，兩邊相差在 10bp 以內

●
10,008

個交易日相差超過 1%，全部逐筆歸因

10,008 個不一致的交易日，每一筆都找到原因

兩邊日報酬相差超過 1% 的交易日數 (上市櫃)



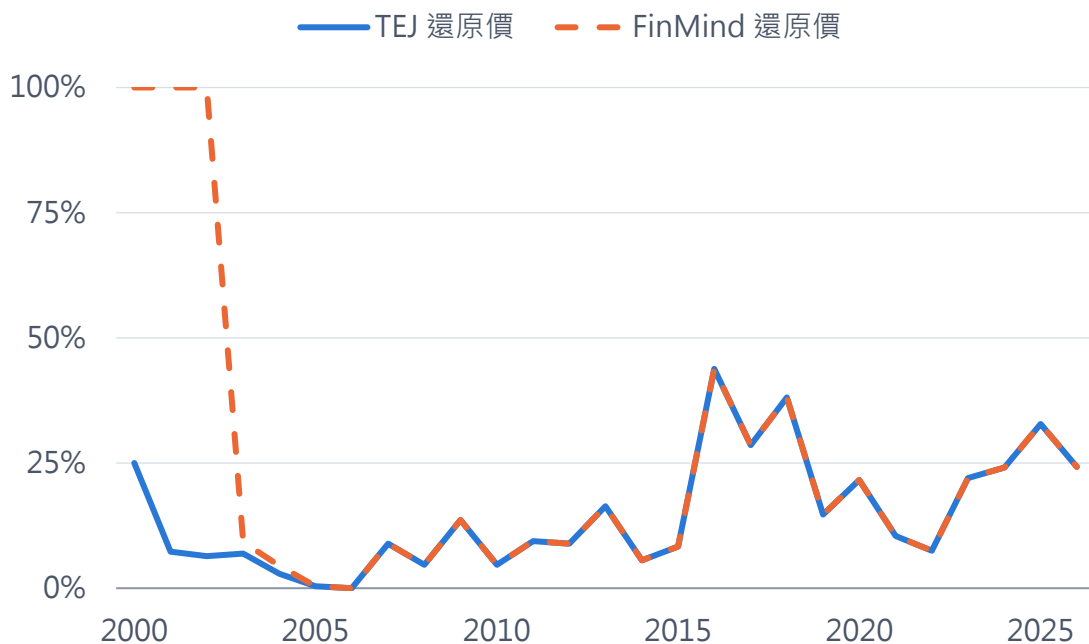
本專案管線造成的差異：修正前 105 天，修正後 10 天 (都落在最近一次每週重抓之後，下次重抓即對齊)。

- **FinMind 未還原除權息**
2,806 天、864 檔，幾乎都在 2007 年以前。
- **無成交日的記法不同**
FinMind 把無成交日記成 0、還原價沿用前一日；TEJ 記參考價，其中 1,614 天正好落在漲跌停價。
- **FinMind 多調**
20 天：未還原價沒有事件，還原價卻跳動，例如 6949 在 2024-03-11 為 -95%。
- **兩邊都調、幅度不同**
9 天，尚待逐筆查證。

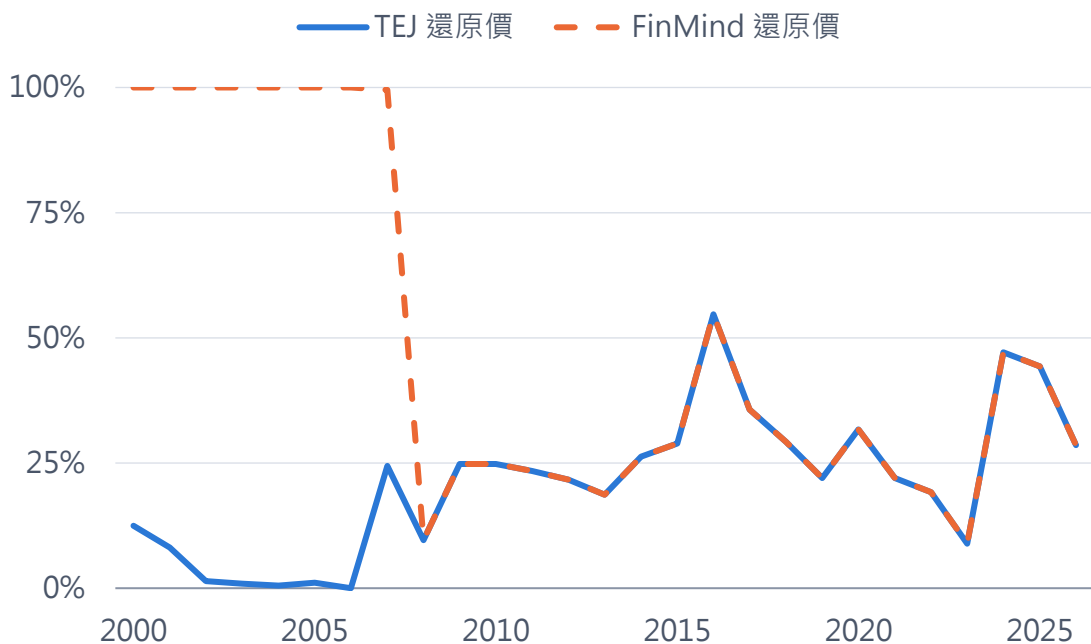
發現一：FinMind 的還原價，早期其實沒有還原

每年「未還原價超過漲跌幅」的跳空中，還原後仍然留著的比例。100% 表示完全沒有調整。

上市



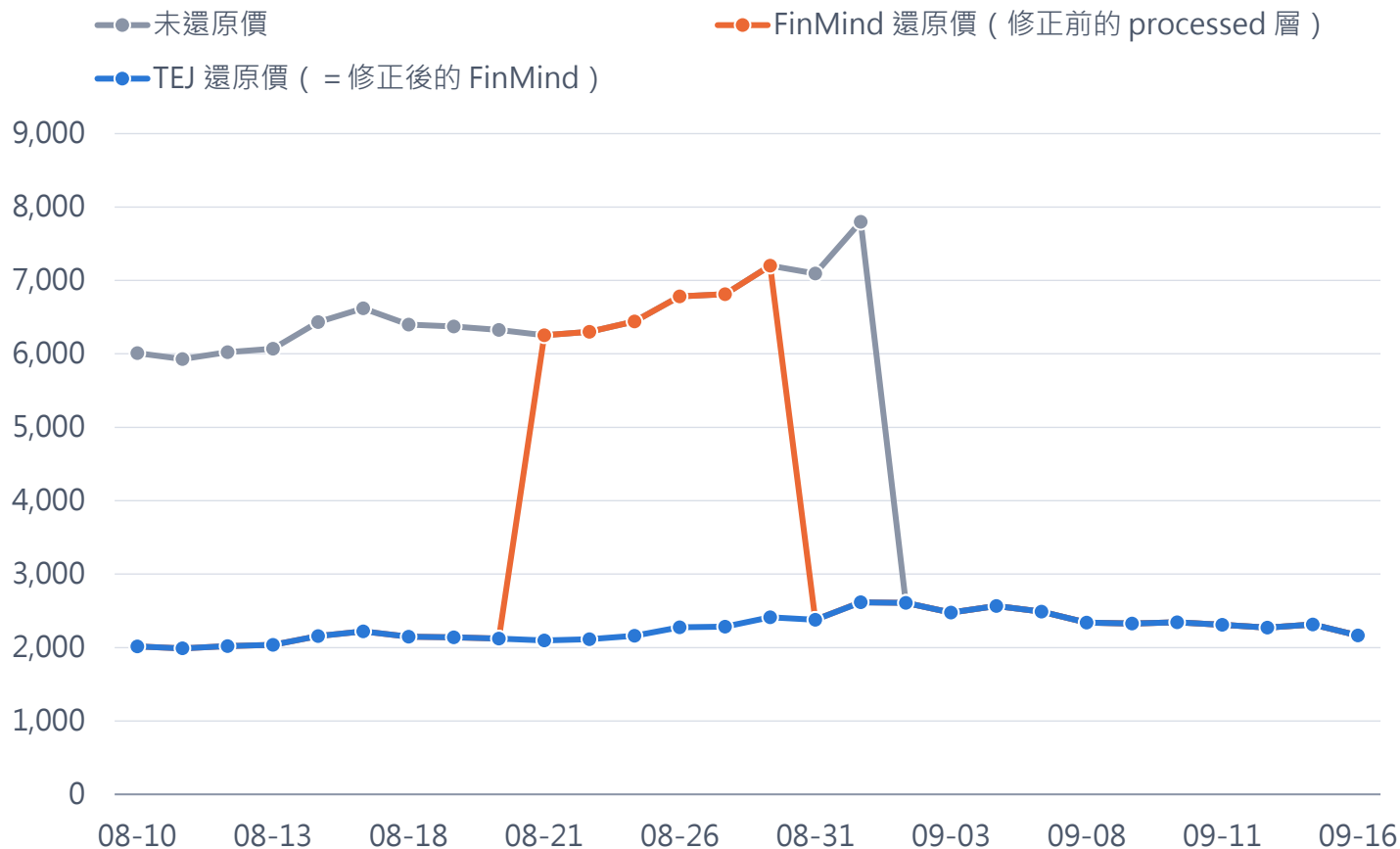
上櫃



上市股到 2002 年、上櫃股到 2007 年，FinMind 的還原價留著全部的除權息跳空；之後兩條線重合。共 2,806 個交易日、864 檔——拿這段期間做回測，除權息會被當成暴跌。

發現二：交叉驗證抓到自己管線的錯誤

6669 緯穎在 2026-09-02 除權。修正前，本專案的還原價在前一週出現 +195% 與 -67% 的假報酬。



原因

去重時讓「全市場日檔」蓋過「ticker 全歷史」。對未還原價這是對的；但還原價的日檔只是抓取當下那個基準的快照，每週重抓的正確歷史反而被舊快照蓋回去。

修正

還原價改為 ticker 優先，日檔只補 ticker 檔尚未涵蓋的日子；新增 4 個測試與 1 條健康檢查。



319 檔

1,046 列被舊基準污染



19 → 6 天

混合基準的交易日數

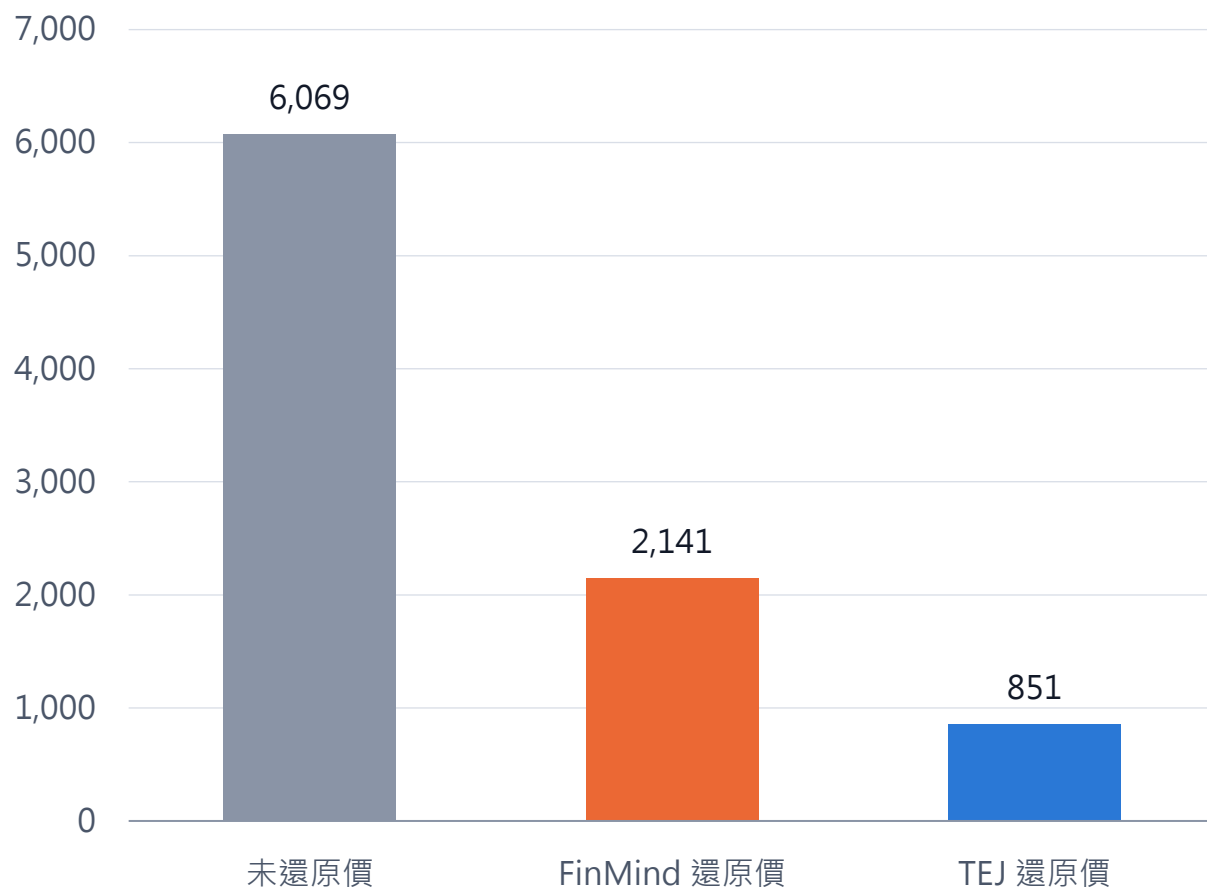
發現三：只有對照第二個來源才看得到的缺陷

現象	規模（實測）	影響
無成交日的收盤價記成 0	FinMind 未還原價 387,319 列（3.3%），其中 36,489 列仍有成交量	直接算報酬會得到 -100%；查詢時要排除
成交量單位不一致	2000–2003 年有 15–27% 的列，成交量恰為 TEJ 的千分之一	早期成交量不能直接跨年比較
上櫃資料缺漏	2007 年 1–4 月，上櫃股幾乎整段沒有資料	這段期間的橫斷面不完整
減資後復牌未調整	818 個停牌後跳空超過 25% 的事件：TEJ 消除 662 個，FinMind 405 個	FinMind 有 413 個完全沒有調整
多餘的調整	20 個交易日：未還原價沒有事件，還原價卻跳動	屬上游錯誤，已列出清單

以上都出在 FinMind 一側；TEJ 一側的疑點見「兩個來源各自不能直接相信的地方」。

誰比較可信：用漲跌幅限制檢驗還原後的報酬

同一批 867 萬筆日報酬中，超過當時漲跌幅限制的筆數



- **TEJ 留下的較少。**還原後 TEJ 剩 851 筆、FinMind 剩 2,141 筆；剩下的多是真實事件，例如新掛牌前五日沒有漲跌幅限制。
- **2008 年以後兩者幾乎相同。**2,074 檔中有 1,935 檔，整段累積報酬相差不到 1%。
- **全期間差距來自早期。**2,101 檔中有 775 檔累積報酬相差超過 10%，來源是 2008 年前沒有還原的除權息。

● 結論

算報酬、做回測用 TEJ；要全市場廣度、真實成交價與 2000 年以前的歷史，用 FinMind 的未還原價。

兩個來源各自不能直接相信的地方

● FinMind

- 還原價：上市股 2003 年前、上櫃股 2008 年前沒有還原；已下市公司幾乎沒有還原價
- 未還原價：無成交日記成 0；2000–2003 年成交量單位不一致；2007 年初上櫃缺漏
- 2004-02-11 以前只有現存公司；官方交易日曆含實際休市的日子
- 維度表每次重爬都會改日期戳記，產業分類沒有歷史版本

● TEJ

- 同一列混有兩種尺度：只有開高低收是還原價
- 18 個交易日的還原報酬超過漲跌幅，而未還原價與 FinMind 都沒有，原因待查
- 內建的報酬率欄位與還原收盤價算出的報酬，有 2,788 列相差超過 1%
- 67,707 筆存續期內的缺漏日尚未逐檔查證
- 2000-09-04 以前下市的公司不在內；raw 無法重製

四 限制與收穫

還沒做到的事，以及這個專案練到的能力

限制與下一步

● 目前的限制

- FinMind 排程自 2026-09-19 起因 token 降級而停擺，資料停在 9 月 16 日
- TEJ 每日排程尚未註冊，指數成分以外的股票最多落後一週
- TEJ 一側的異常報酬與報酬欄位差異，還沒有找到解釋
- 只有日頻價量與指數成分，沒有財報、法人與融資券資料

● 下一步

- 用未還原價與除權息事件自行計算還原因子，補上 FinMind 缺的早期還原與已下市公司
- 把跨來源比對變成每週自動執行的檢查，差異直接進對帳 view
- 逐檔查證 TEJ 的缺漏日與異常報酬
- 加入財報與籌碼資料，支援因子研究

資料先對，模型才有意義

這個專案練到的三種能力

● 資料工程

分層契約、原子寫入與暫存目錄換檔、跨行程限流、以 COM 自動化一個沒有 API 的來源、可重跑的排程與自我修復。

● 統計與驗證

以報酬而非水準比較兩個來源、用交易制度做合理性檢驗、把一萬個差異逐筆歸因、量化存活者偏誤。

● 研究紀律

數字一律實測、可以重現；找到自己的錯誤就修正並留下紀錄；每個結論都寫明適用範圍與限制。

完整說明見同資料夾的《financial-db 專案說明書》；互動版資料地圖與 Schema 圖的連結在 README.md。